La sintaxis de JavaScript

Índice

- La sintaxis de JavaScript
  - Índice
  - Convenciones de nombres
    - Nombres descriptivos
    - Índices y contadores
    - Constantes, clases y variables
    - Tipos de datos
      - Ejemplos
    - Declaración de variables
    - Ámbito de las variables
    - Ámbito de las variables. Hoisting
    - Ámbito de las variables. Declaración con var
    - Ámbito de las variables. Declaración con let
  - Operadores
    - Aritméticos
    - Ternario
    - Concatenación de strings
    - Operadores lógicos
    - Bitwise
    - Operaciones de comparación
  - Propiedades globales y funciones globales
  - Conversión de tipos
  - Arrays
    - Declaraciónes de arrays
    - Funciones predefinidas típicas de Arrays
      - Ejemplos de funciones de arrays
      - Resultado de la ejecución
      - Uso de sort() para ordenar arrays
  - Control de flujo
  - Strings
    - Funciones predefinidas típicas de strings
  - Date
    - Funciones típicas
  - Template Strings

Convenciones de nombres

Nombres descriptivos

- Una mala costumbre habitual cuando se empieza en la programación es darle un nombre muy poco descriptivo
- A medida que el código crece, se vuelve insostenible
- Evita nombres poco claros o inconsistentes como tmp, a, b2, variable2, etc

Índices y contadores

- Cuando trabajamos en bucles for (o bucles en general)
- El ámbito de una variable que actúa como contador (índice) es muy reducido (esa variable solo existe y afecta al interior del bucle)
- Las variables que actúan como contador suelen nombrarse con una letra minúscula empezand desde i: `i`, `j`, `k`, etc. A veces también se usan letras como `a`, `b`, `c`... o la inicial minúscula de lo que representan: `c` para un contador, `p` para una posición, etc.

Constantes, clases y variables

- Las **constantes** son variables que no varían su valor a lo largo del programa. Deben ir siempre en **MAYÚSCULAS**
- Las **clases** son estructuras de código más complejas. Los nombres de las clases se escriben en PascalCase.
- Las variables, por último siempre en **camelCase**

Tipos de datos

- JS es un lenguaje de **tipado débil** porque cuando se declara una variable no se indica su tipo.
- JS es un lenguaje de **tipado dinámico** porque las variables pueden cambiar de tipo en tiempo de ejecución.
- Existen dos tipos de datos:
  - **Primitivos** (inmutables)
    - **Boolean**
    - **Number** (64 bits)
    - **String**
    - **Null**
    - **Undefined**
  - **Objetos** (mutables, todo lo que no sean primitivos, incluyendo funciones)

Ejemplos

```
let numero1=1;        console.log(typeof numero1);     //number
let numero2=1.03;     console.log(typeof numero2);     //number
let booleano = true; console.log(typeof booleano);    //boolean
let cadena="Hola";    console.log(typeof cadena);      //string
let nulo=null;        console.log(typeof nulo);        //object
                      console.log(nulo);               //null
let indefinido;       console.log(typeof indefinido);  //undefined
                      console.log(indefinido);         //undefined
let objeto = {a:1};   console.log(typeof objeto);      //object
```

El operador "typeof" devuelve el tipo de dato de una variable declarada.

```
let a=123;
a="Hola";
console.log(a); // Hola
```

El tipo de la variable puede cambiar en tiempo de ejecución sin problemas.

En algunos casos, typeof resulta insuficiente porque en tipos de datos más avanzados simplemente nos indica que son objetos.

Con `constructor.name` podemos obtener el tipo de constructor que se utiliza.

Ejemplos:

```
console.log(text.constructor.name) // String
console.log(number.constructor.name) // Number
console.log(boolean.constructor.name) // Boolean
console.log(notDefined.constructor.name) // ERROR, sólo funciona con
variables definidas
```

Declaración de variables

- Se pueden declarar variables de tres formas
  - **Directamente (`a = 2`)**. Es la opción menos recomendada. A partir de ES5 se recomienda usar el modo estricto escribiendo en la primera línea del fichero "use strict"; con lo que se impide declaraciones directas.
  - **Utilizando `var`** : permite declarar una variable cuyo ámbito es la función donde se define o global (si se define fuera de una función)
  - **Utilizando `let`** : es la opción recomendada. La definición tiene ámbito de bloque de código, de función o global si se define en la raíz del documetno. Disponible a partir de ES6.
- Las constantes se definen con `const` y tienen ámbito de bloque.

Ámbito de las variables

```
let a1=1;           // Ámbito global
//...
{                   // Ámbito de bloque
    let a2=2;
    //...
}
function f() {      // Ámbito de función
    let a3=3;
    //...
    if (true){      // Ámbito de bloque
        let a4=4;
        //...
    }
}
```

Las funciones **IIFE** (Immediately-invoked function expressions) o **funciones autoinvocadas** permiten encapsular variables declaradas mediante `var` creando un ámbito local

```
(function(){
    //....
}());
```

```
(function(n){
    var saludo="Hola"
    console.log(saludo+n) // Hola Juan
}("Juan"))
```

Ámbito de las variables. Hoisting

```
console.log(a); //undefined
var a=1;
{
    console.log(a); // 1
}

(function(){
    console.log(a); //1
    console.log(b); //undefined
    var b=2;
    console.log(b); //2
}());
console.log(b); //Error: b is not defined
```

```
console.log(a); //Error: a is not defined
let a=1;
{
    console.log(a); // 1
}

(function(){
    console.log(a); // 1
    console.log(b); //Error: b is not defined
    let b=2;
    console.log(b); // 2
}());
console.log(b);  //Error: b is not defined
```

Ámbito de las variables. Declaración con var

```
var a=1;
console.log(1); // 1
{
    console.log(a); // 1
}
(function f(){
    console.log(a); // 1
    if (true){
        console.log(a); // 1
    }
})
```

```
var a=1;
console.log(1); // 1
(function f(){
    var a=2;
    console.log(a) // 2
    if (true) {
        var a=3;
        console.log(a); // 3
    }
    console.log(a); // 3
})();
console.log(a); // 1
var a=5;
console.log(a); // 5;
```

Ámbito de las variables. Declaración con let

```
let a=1;
console.log(1); // 1
{
    console.log(a); // 1
}
(function f(){
    console.log(a); // 1
    if (true){
        console.log(a); // 1
    }
    console.log(a); //1
})();
```

```
let a = 1;
console.log(1); // 1
(function f(){
    let a = 2;
    console.log(a); // 2
    if (true){
        let a=3;
        console.log(a); // 3
    }
    console.log(a); // 2
})();
console.log(a); // 1
let a = 5; // Error: already declared
```

Operadores

Aritméticos

- Suma: a+b
- Resta: a-b
- Multiplicación: a*b
- División: a/b
- Resto de división: a%b
- Exponenciación: a**b
- Incremento: ++
- Decremento: --

Ternario

```
(condicion)? instruccion1: instruccion2
```

Concatenación de strings

```
"string1" + "string2"
```

Operadores lógicos

- AND: `a&&b`
- OR: `a||b`
- NOT: `!a`

Bitwise

- AND: `a&b`
- OR: `a|b`
- XOR: `a^b`
- NOT: `~a`
- Shift Left: `a<<b`
- Shift Right: `a>>b`

Operaciones de comparación

- `>, <, >=, <=, ==, !=`
- `===` (igualdad estricta)
- `!==` (desigualdad estricta)

Propiedades globales y funciones globales

- `Infinity` : Representa el valor infinito
- `NaN` : Not a number
- `null` : valor nulo
- `isFinite()` , `isNaN()` , `parseFloat()` , `parseInt()`

Conversión de tipos

- Se puede cambiar el tipo de las variables dinámicamente (incluso en el modo "use strict").
- Para convertir entre tipos se pueden utilizart los métodos:
  - parseInt()
  - parseFloat()
  - toString()
  - eval()

Cuando hacemos una asignación o como resultado de una expresió nhay que tener en cuenta la `coerción de tipos` .

```
"use strict"
let a=1;
a="hola"
console.log(a); // hola
let b = parseInt("123.45");
console.log(b) // 123
let c = parseFloat("123.45");
console.log(c) // 123.45
let d = 46;
console.log(d.toString(2)); // 101110
console.log(d.toString(16)); // 2e
let e=eval("2+4+parseInt(1.2)");
console.log(e); // 7
```

Arrays

Declaraciónes de arrays

```
let lista1 = new Array();
let lista2 = Array();
let lista3 = [];

lista1[0] = 1;
console.log(lista1[0]);
lista1[5] = "Pepe";

// For clásico
for (let i = 0; i<lista1.length; i++){
    console.log(i, lista1[i]) // 0 1, 1 undefined, 2 undefined, 3 undefined,
4 undefined, 5 Pepe
}

// Foreach: ignora los undefined
lista1.forEach((element, index) => console.log(index, element)); // 0 1, 5
Pepe


lista1["pos1"] = "Laura";
lista1.pos2 = "Santi";

// For clásico
for (let i = 0; i<lista1.length; i++){
    console.log(i, lista1[i]) // 0 1, 1 undefined, 2 undefined, 3 undefined,
4 undefined, 5 Pepe
}

// Foreach: ignora los undefined
lista1.forEach((element, index) => console.log(index, element)); // 0 1, 5
Pepe

console.log(lista1.pos1) // Laura
console.log(lista1["pos2"]) // Santi
```

Funciones predefinidas típicas de Arrays

- `length` : propiedad que devuelve el número de elementos
- `concat()` : permite concatenar varios arrays
- `join()` : une los elementos de un array y los almacena en un string
- `pop()` : extrae el último elemento de un array
- `push()` : añade un elemento al final del array
- `shift()` : extrae el primer el emento de un array
- `unshift()` : añade un elemento al comienzo del array
- `reverse()` : invierte la posición de los elementos del array
- `sort()` : ordena un array
- `indexOf()` : devuelve la posición de un elemento en el array
- `lastIndexOf()` : devuelve la posición final de un elemento en el array
- `slice()` : devuelve un nuevo array con parte de los elementos del array inicial
- `splice()` : cambia el contenido de un array: inserta, reemplaza o borra elementos del array

Ejemplos de funciones de arrays

```
let array1 = []
let array2 = []

array1.push("Hola")
array1.push("Adiós")
array2.push("Hola2")
array2.push("Adiós2")

console.log("Array 1:", array1.join(" / "))
console.log("Array 2:", array2.join(" / "))

let array3 = array1.concat(array2)
console.log("Array 3:", array3.join(" / "))

array3.pop()
console.log("Array 3 después de un pop:", array3.join(" / "))

array3.shift()
console.log("Array 3 después de un shift:", array3.join(" / "))

array3.unshift("Hola3")
array3.unshift("Adiós3")
console.log("Array 3 después de dos unshift:", array3.join(" / "))

array3.reverse()
console.log("Array 3 después de un reverse:", array3.join(" / "))

array3.sort()
console.log("Array 3 después de un sort:", array3.join(" / "))

array3.push("Hola2")
console.log("Posición de 'Hola2' en el array3:", array3.indexOf("Hola2"))
console.log("Posición final de 'Hola2' en el array3:",
array3.lastIndexOf("Hola2"))

array4 = array3.slice(1, 4)
console.log("Array 4 (slice de array3):", array4.join(" / "))

console.log("Array 1 antes de un splice:", array1.join(" / "))
array1.splice(1, 1, "Hola333", "Adiós1")
console.log("Array 1 después de un splice:", array1.join(" / "))

array1.sort((a, b) => a.length - b.length)
console.log("Array 1 después de un sort por longitud:", array1.join(" / "))
```

Resultado de la ejecución

> *Array 1: Hola / Adiós*
> *Array 2: Hola2 / Adiós2*
> *Array 3: Hola / Adiós / Hola2 / Adiós2*
> *Array 3 después de un pop: Hola / Adiós / Hola2*
> *Array 3 después de un shift: Adiós / Hola2*
> *Array 3 después de dos unshift: Adiós3 / Hola3 / Adiós / Hola2*
> *Array 3 después de un reverse: Hola2 / Adiós / Hola3 / Adiós3*
> *Array 3 después de un sort: Adiós / Adiós3 / Hola2 / Hola3*
> *Posición de 'Hola2' en el array3: 2*
> *Posición final de 'Hola2' en el array3: 4*
> *Array 4 (slice de array3): Adiós3 / Hola2 / Hola3*
> *Array 1 antes de un splice: Hola / Adiós*
> *Array 1 después de un splice: Hola / Hola333 / Adiós1*
> *Array 1 después de un sort por longitud: Hola / Adiós1 / Hola333*

Uso de sort() para ordenar arrays

- Sort ordena el array convirtiendo cada elemento en caracteres Unicode. Ejemplo:

```
console.log(['verde' , 'azul' , 'rojo'].sort()); // ['azul', 'rojo',
'verde']
console.log([20,1,2,10].sort()) // [1,10,2,20]
console.log([6,-2,2, -7].sort()) // [-2,-7,2,6]
```

- Si se desea ordenar un array con números (o cualquier otro objeto) correctamente se debe pasar una función callback como parámetro con el criterio de comparación. Ejemplo:

```
console.log([6,-2,2,-7].sort(function (a,b){
    return a-b;
}));
```

- Las funciones callback anónimas pueden simplificarse mediante lambdas

```
console.log([6,-2,2,-7].sort((a, b) => a - b);
```

Control de flujo

- Las intstrucciones se pueden agrupar en bloques entre los caracteres `{` y `}`.
- El control de flujo se realiza con instrucciones similares a otros lenguajes como Java o C#.

```
if(expresion){
    //...
} else {
    //...
}
```

```
switch(var){
    case valor1:
        //...
        break;
    case valor2:
        //...
        break;
    //...
    default:
        //...
        break;
}
```

```
    do{
        //...
    } while(condición)
```

```
    while(condición){
        //...
    }
```

```
    for (let i=0; i<length; i++){
        //...
    }
```

```
    for (indice in array){
        console.log(indice, "=>", array[indice])
    }
```

```
    for (objeto of array){
        console.log(objeto)
    }
```

```
    for (iterator of array){
        //...
    }
```

```
try {

} catch {

}
```

Strings

Funciones predefinidas típicas de strings

- `length` : propiedad que devuelve la longitud de la cadena
- `concat()` : concatena dos strings
- `charAt(pos)` : devuelve el caracter en la posición `pos`
- `charCodeAt(p)` : devuelve el código UNICODE de la posición `p`
- `indexOf(sub)` : devuelve la primera posición de la subcadena `sub`
- `lastIndexOf(sub)` : devuelve la última posición de la subcadena `sub`
- `substring(i,f)` : devuelve la subcadena desde la posición i a la posición `f`
- `toLowerCase()` : convierte la cadena a minúsculas
- `toUpperCase()` : convierte la cadena a mayúsculas
- `split(c)` : divide una cadena en diferents partes separadas por el carácter `c` y las devuelve en un array.

Date

Funciones típicas

- Constructor. Tiene varias sobrecargas:
  - `new Date()`
  - `new Date(millis)`
  - `new Date(cadenaFecha)`
  - `new Date(año, mes, dia[hora, minuto, segundo, mils])`
- Funciones:
  - `now()` : Devuelve el valor numérico correspondiente a la fecha y hora actual.
  - `parse()` : Parsea una string tipo `aaaa-mm-dd` y devuelve una fecha
  - `getFullYear()` : Devuelve el año de la fecha
  - `getMonth()` : Devuelve el mes
  - `getDate()` : Devuelve el día del mes
  - `getDay()` : Devuelve el día de la semana (0=domingo, 6=sábado)
  - `getHours()` , `getMinutes()` , `getSeconds()` , `getMilliseconds()`
  - `getTime()` : Devuelve el valor numérico (timestamp) en ms correspondiente a la fecha
  - Existen setters para modificer la fecha.

Template Strings

Los Template Strings, introducidos en ES6, permiten funcionalidad adicional a los strings clásicos, por ejemplo, interpolación de strings, escribir una cadena en varias líneas o `Tagged Templates` (plantillas etiquetadas)